

HoTHP: Hyperbolic Rotary Position Embedding-based Transformer Hawkes Process

Extrapolação de comprimento em Processos Pontuais Temporais

Hugo Ramos Soares

Mestrado - Defesa Final, 28 de agosto de 2026

Universidade Federal do Ceará

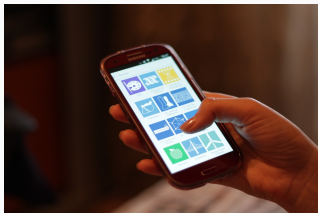
Orientador: César Lincoln C. Mattos (UFC)

Coorientador: João Paulo Pordeus Gomes (UFC)

Banca: José Antonio Fernandes de Macêdo (UFC)

Diego Parente Paiva Mesquita (FGV)

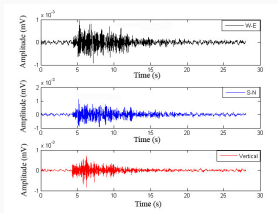
Tudo acontece no tempo



Uma mensagem chega.
Depois outra.



Um paciente é atendido,
e volta meses depois.

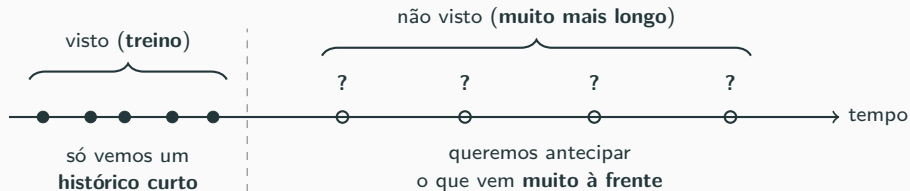


A terra treme.
Quando vai tremer de novo?

A pergunta por trás de todos eles:
quando acontece o próximo?

Imagens: Wikimedia Commons.

Aprender com pouco, prever muito à frente



O desafio deste trabalho

Treinar em sequências **curtas** de eventos e continuar confiável em sequências **muito mais longas**.

Introdução

Fundamentação: Processos Pontuais Temporais

Modelos neurais e atenção

Trabalhos relacionados: extrapolação de comprimento

Metodologia: HoTHP

Experimentos e resultados

Conclusão

Introdução

Muitos fenômenos acontecem em tempo contínuo e precisamos modelá-los:

Telemetria

falhas de sistema, alertas de infraestrutura

Redes sociais

retweets, curtidas, cascatas de respostas

Saúde

internações, exames, prescrições de um paciente

Manutenção

falhas em equipamentos industriais

O que eles têm em comum?

Cada domínio produz uma sequência ordenada de eventos com instantes de tempo:

$$\mathcal{S} = \{(t_1, k_1), (t_2, k_2), \dots, (t_N, k_N)\}$$

onde t_i é o tempo e k_i é o tipo (marca) do evento i .

O que queremos modelar?

Dados os eventos passados, queremos responder:

- **Quando** o próximo evento vai acontecer?
- **De qual tipo** ele será?

A ferramenta central é a **intensidade condicional**:

$$\lambda(t \mid \mathcal{H}_t) = \lim_{\Delta t \rightarrow 0} \frac{P(\text{evento em } [t, t + \Delta t) \mid \mathcal{H}_t)}{\Delta t}$$

onde $\mathcal{H}_t$ é o histórico de todos os eventos antes de t .

Objetivo de aprendizado

Aprender $\lambda(t)$ a partir dos dados de modo que o modelo generalize para sequências com comprimentos diferentes dos vistos no treino.

O problema prático: treinar curto, testar longo

- Coletar sequências longas de eventos costuma ser caro ou inviável.
- Queremos **treinar em sequências curtas** e rodar o modelo em sequências **arbitrariamente mais longas** [5].
- Modelos de processos pontuais baseados em Transformer degradam bastante quando a sequência de teste é mais longa que a de treino. A capacidade de se manter estável nesse regime é chamada de *extrapolação de comprimento*.

Contribuição deste trabalho: HoTHP

Um modelo da família Transformer que embute um **viés de recência exponencial** (eventos recentes pesam mais), espelhando a estrutura do processo de Hawkes, e por isso mantém a verossimilhança estável quando a sequência de teste é muito mais longa que a de treino. (*Como esse viés entra na atenção, vemos na metodologia.*)

Pergunta de pesquisa

No mecanismo de atenção (detalhado adiante), substituir um kernel que **oscila** por um que **decai exponencialmente** melhora a capacidade de extrapolação de comprimento dos processos pontuais temporais neurais?

Para responder, nós:

- propomos e implementamos o **HoTHP**, adaptando o HoPE de posições discretas para instantes de tempo contínuos;
- avaliamos *sob treinar curto, testar longo* em dados **sintéticos** (processo gerador conhecido) e dados **reais** de domínios variados;
- analisamos *sob quais condições* o viés de recência é benéfico, neutro ou prejudicial (intensidades aprendidas, acurácia, RMSE e uma ablação de escala temporal).

Fundamentação: Processos Pontuais Temporais

A pergunta que ela responde: quantos eventos veremos em uma janela fixa, se eles acontecem de forma independente a uma taxa média constante?

Definição [6]

Se o *número esperado* de eventos na janela é $\lambda > 0$, a probabilidade de observar exatamente k eventos é

$$\Pr(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

- X : uma **variável aleatória**, a contagem de eventos na janela (desconhecida antes de observarmos);
 - k : um **valor concreto** que essa contagem pode assumir ($0, 1, 2, \dots$);
 - $\Pr(X = k)$: a **probabilidade** de a contagem observada X ser exatamente k .
-
- λ é ao mesmo tempo a **média** e a **variância** da contagem.

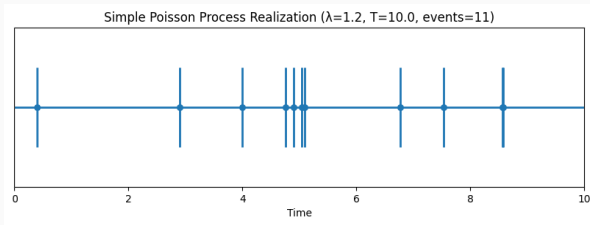
O processo de Poisson

Agora deixe os eventos acontecerem ao longo do tempo, a uma **taxa** constante λ (eventos por unidade de tempo), e seja $N(T)$ a contagem deles até o tempo T [6]. De forma resumida:

- eventos em intervalos curtos e **sem sobreposição** acontecem de forma **independente**;
- cada intervalo curto recebe **no máximo um** evento, com chance proporcional a λ .

Então $N(T)$ é Poisson com média λT (taxa vezes tempo):

$$\Pr\{N(T) = n\} = \frac{(\lambda T)^n}{n!} e^{-\lambda T}$$



Modelos probabilísticos para sequências de eventos discretos em **tempo contínuo** [10]. Eles lidam com eventos em tempos irregulares e arbitrários.

- Totalmente descritos pela intensidade condicional $\lambda(t \mid \mathcal{H}_t)$.
- Com **marcas** (tipos de evento), a sequência é $X = \{(t_1, m_1), \dots, (t_N, m_N)\}$ e usamos K intensidades:

$$\lambda_k(t \mid \mathcal{H}_t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(\text{evento do tipo } k \text{ em } [t, t + \Delta t] \mid \mathcal{H}_t)}{\Delta t}$$

Sobre a palavra “marca”

Usamos *marca* e *tipo* como sinônimos: a marca é um tipo de evento categórico, uma dentre K classes.

Dados todos os eventos observados em $[0, T]$, os TPPs são treinados maximizando a log-verossimilhança [4]:

$$\mathcal{L} = \sum_{i: t_i \leq T} \log \lambda_{k_i}(t_i) - \underbrace{\int_0^T \bar{\lambda}(t) dt}_{\Lambda}, \quad \bar{\lambda}(t) = \sum_{k=1}^K \lambda_k(t)$$

- Primeiro termo: recompensa uma **intensidade alta** nos momentos em que os eventos realmente ocorreram (e para o tipo correto k_i).
- Segundo termo (o *compensador* Λ): penaliza massa de intensidade onde nenhum evento ocorreu.
- O treino minimiza a log-verossimilhança **negativa** (NLL), nossa principal métrica de avaliação.

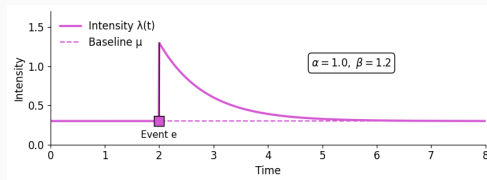
Na prática, a integral é estimada numericamente, exatamente da mesma forma para todos os modelos que comparamos.

Poisson supõe eventos independentes. O **processo de Hawkes** supõe que **eventos passados aumentam a probabilidade de eventos futuros**, com uma influência que decai exponencialmente [3]:

$$\lambda(t \mid \mathcal{H}_t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}$$

- μ : intensidade de base.
- α : salto instantâneo causado por um evento.
- β : quão rápido essa influência decai.

O decaimento exponencial é um **viés de recência**: eventos recentes importam mais.



Formas cada vez mais ricas de especificar a intensidade:

1. **Poisson homogêneo:** a intensidade λ é constante.
2. **Poisson não homogêneo:** $\lambda(t)$ varia no tempo, mas de forma determinística.
3. **Processo de Cox** (Poisson duplamente estocástico): $\lambda(t)$ é ele próprio um processo *aleatório*, guiado por fatores externos aos eventos.
4. **Processo de Hawkes:** $\lambda(t)$ é aleatório porque depende dos *eventos passados do próprio processo*, cada um elevando temporariamente a intensidade.

Por que isso importa aqui

O Hawkes também tem intensidade estocástica, mas, diferente do Cox, sua aleatoriedade é **endógena**: $\lambda(t)$ é um funcional do próprio passado do processo (um processo *auto-excitante*, não duplamente estocástico). É exatamente essa dependência do histórico, com decaimento exponencial, que motiva o viés de recência do nosso modelo.

Modelos neurais e atenção

O processo de Hawkes clássico é limitado: a influência é sempre excitatória, constante por tipo, e decai exponencialmente. Mei e Eisner [4] relaxam essas suposições com redes neurais.

- Permitem $\alpha, \mu \in \mathbb{R}$, de modo que os eventos podem **inibir** além de excitar; um **softplus** mantém a intensidade positiva.
- Substituem a soma aditiva por uma **LSTM de tempo contínuo** (CT-LSTM): a célula de memória decai exponencialmente entre eventos.

$$\lambda_k(t) = f_k\left(\mathbf{w}_k^\top \mathbf{h}(t)\right), \quad \mathbf{c}(t) = \bar{\mathbf{c}} + (\mathbf{c} - \bar{\mathbf{c}}) e^{-\delta(t-t_i)}$$

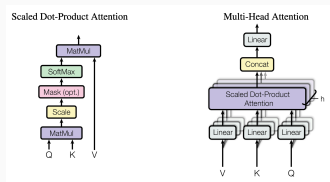
Um modelo simples, mas forte: ainda é um dos melhores benchmarks hoje, e uma baseline importante nos nossos experimentos.

Transformers e o mecanismo de atenção

RNNs (incluindo a CT-LSTM) processam eventos **sequencialmente**, o que é lento e tem dificuldade com sequências longas. O Transformer [8] elimina a recorrência: todos os eventos são processados de uma vez pela **atenção**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Cada evento presta atenção a todos os outros; os pesos medem a relevância.
- A atenção **multi-cabeça** roda h projeções em paralelo e as concatena.

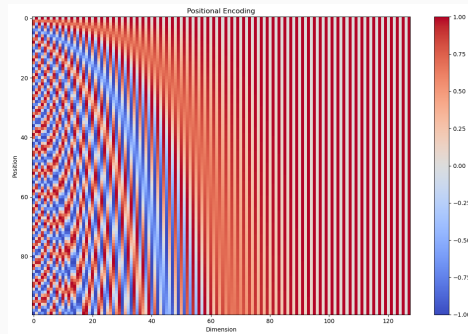


Fonte: [8]

Codificação posicional: a atenção não tem ordem

A atenção é **equivariante a permutações**: reordenar as entradas apenas reordena as saídas, então, sozinha, ela não conhece a ordem nem o tempo dos eventos. A posição precisa ser injetada.

- **Transformer original**: soma uma codificação senoidal da *posição discreta* a cada embedding de token.
- Cada dimensão é um seno/cosseno de uma frequência diferente: rápida à esquerda, lenta à direita, com assinatura única por posição.
- **Para TPPs** precisamos do *instante* de fato, não só da ordem, já que o momento do evento importa.



Codificação senoidal: posição (linhas) vs. dimensão (colunas).

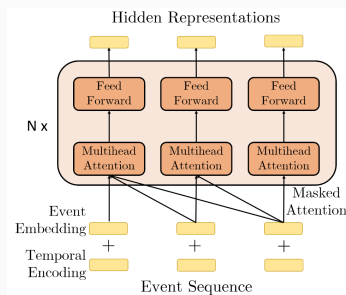
Transformer Hawkes Process (THP)

Zuo et al. [10] adaptam o Transformer para TPPs com uma **codificação temporal** que usa o instante t_j em vez de uma posição discreta:

$$[z(t_j)]_i = \begin{cases} \cos\left(t_j/10000^{(i-1)/M}\right), & i \text{ ímpar}, \\ \sin\left(t_j/10000^{i/M}\right), & i \text{ par}. \end{cases}$$

A codificação é somada ao embedding de tipo, e então passa por autoatenção e uma rede feed-forward. A intensidade é

$$\lambda_k(t) = f_k\left(\underbrace{\alpha_k \frac{t-t_j}{t_j}}_{\text{atual}} + \underbrace{\mathbf{w}_k^\top \mathbf{h}(t_j)}_{\text{histórico}} + \underbrace{b_k}_{\text{base}}\right).$$



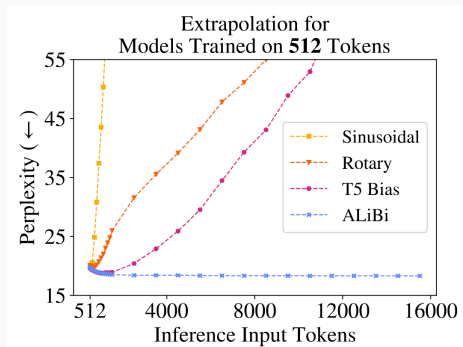
Fonte: [10]

Trabalhos relacionados: extrapolação de comprimento

Extrapolação de comprimento em modelos de linguagem

Um modelo treinado em sequências de comprimento L_{train} deveria se manter estável quando testado em $L_{\text{test}} > L_{\text{train}}$ [5].

- Posições senoidais absolutas produzem padrões de codificação nunca vistos no treino para índices maiores: a perplexidade explode.
- Codificações rotativas (RoPE) ajudam ao usar posições *relativas*, mas Press et al. [5] mostraram que elas ainda falham para sequências muito mais longas.
- Só o **ALiBi**, com uma penalidade linear de distância, se mantém estável bem além do comprimento de treino.



Fonte: [5]

Adaptamos o protocolo *treinar curto, testar longo* [5] para processos pontuais, usando o **número de eventos** como comprimento.

- Treinar em sequências de comprimento fixo (por exemplo, $L = 50$ eventos).
- Testar em sequências α vezes mais longas, $\alpha \in \{1, 2, 5, 10\}$.
- Em $\alpha = 10\times$: treinar com 50 eventos, testar com 500.

O que medimos

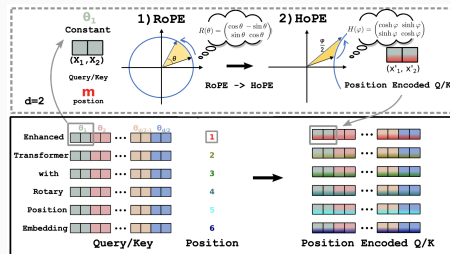
Se a NLL se mantém estável conforme α cresce. Um modelo com um viés de recência adequado deve degradar muito pouco.

Rotary Position Embedding (RoPE)

Su et al. [7] notaram que a atenção só precisa da posição **relativa**. O RoPE rotaciona pares de dimensões do embedding por um ângulo proporcional à posição:

$$\mathbf{R}_j(\Delta) = \begin{bmatrix} \cos(\theta_j \Delta) & \sin(\theta_j \Delta) \\ -\sin(\theta_j \Delta) & \cos(\theta_j \Delta) \end{bmatrix}, \quad \theta_j = 10000^{-2(j-1)/d}.$$

- Aplicado a **q** e **k** antes do produto interno.
- O score acaba dependendo apenas da diferença de posições.



Adaptado de [7]

Dai et al. [2] trazem o RoPE para TPPs: rotacionar pelo **instante** em vez do índice discreto.

- O score de atenção depende apenas de $\Delta t = t_i - t_j$.
- Isso torna a verossimilhança **invariante no tempo**: deslocar todos os tempos por uma constante σ não altera o resultado.
- Robusto a ruído nos instantes (dessincronização de relógio), comum em dados reais. Novo estado da arte em vários benchmarks de TPP.

Mas há um porém

O kernel rotativo é construído a partir de cos e sin: ele **oscila**. Essa é a limitação que nosso método ataca.

Press et al. [5] seguem um caminho diferente: nenhum embedding, apenas uma **penalidade linear** fixa no score de atenção:

$$\text{softmax}\left(\mathbf{q}_i \mathbf{K}^\top + m \cdot [-(i-1), \dots, -2, -1, 0]\right)$$

- A penalidade cresce com a distância, então tokens distantes são suprimidos. A inclinação m é fixa por cabeça, nunca aprendida.
- Isso é um **viés indutivo em direção à recência**, embutido na arquitetura em vez de aprendido dos dados.

A conexão com o Hawkes

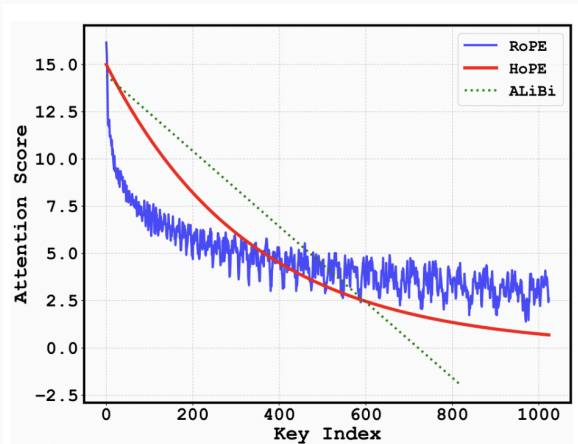
É exatamente a ideia do kernel de Hawkes (a influência $\propto e^{-\beta \Delta t}$ decai com a distância). O THP e o RoTHP abriram mão dessa garantia estrutural ao trocar o kernel por atenção livre. **O HoTHP a devolve.**

Metodologia: HoTHP

A limitação oscilatória do RoTHP

O score do RoTHP depende apenas de $\Delta t = t_i - t_j$, mas seu kernel é construído a partir de $\cos$ e $\sin$, então ele **oscila** em vez de decair.

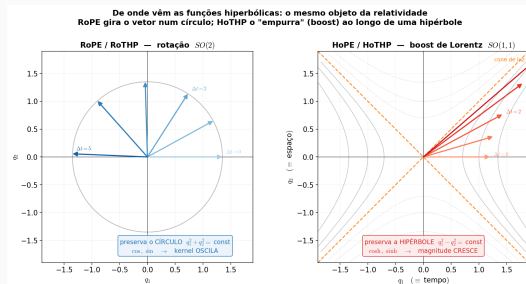
- O **RoPE** (azul) decai apenas *em média*: ruidoso, não monótono, e nunca some de fato.
- Um kernel de Hawkes $\varphi(\Delta t) = \alpha e^{-\beta \Delta t}$ é *estritamente* decrescente. O RoPE viola isso.
- Na extrapolação a fase pode cair em um **pico**: um evento distante recebe score maior que um próximo.



Do kernel trigonométrico ao kernel hiperbólico

Seguindo o HoPE [1], substituímos o bloco de rotação $\mathbf{R}_j$ por um **boost hiperbólico** $\mathbf{B}_j$:

$$\mathbf{R}_j(\Delta t) = \begin{bmatrix} \cos & \sin \\ -\sin & \cos \end{bmatrix} \longrightarrow \mathbf{B}_j(\Delta t) = \begin{bmatrix} \cosh(\theta_j \Delta t) & \sinh(\theta_j \Delta t) \\ \sinh(\theta_j \Delta t) & \cosh(\theta_j \Delta t) \end{bmatrix}$$



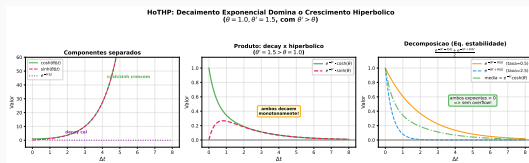
A rotação preserva o **círculo** ($\cos, \sin$ oscilam); o boost de Lorentz preserva a **hipérbole** ($\cosh, \sinh$ são monótonos).

Controlamos esse crescimento com o envelope de decaimento $e^{-|\Delta t|\theta'}$.

O HoPE [1] adiciona um θ' aprendível e amortece $\mathbf{q}, \mathbf{k}$ para que o score decaia. Em tempo contínuo:

$$g(\Delta t) = e^{-|\Delta t| \theta'} \mathbf{q}^\top \mathbf{B}(\theta, \Delta t) \mathbf{k}$$

- Se $\theta' > \max_j \theta_j$, o decaimento domina o crescimento hiperbólico: o score **decai exponencialmente até zero** com $|\Delta t|$, sem oscilação.
- Esse é um viés de recência **estrutural**: vale mesmo para Δt nunca visto no treino. É isso que permite a extrapolação.



Cada cabeça se divide em $d/2$ **pares**. O par p tem frequência θ_p e componentes $(q_1^{(p)}, q_2^{(p)})$ e $(k_1^{(p)}, k_2^{(p)})$.

Score de um par = rotação hiperbólica por Δt

$$s^{(p)} = (q_1 k_1 + q_2 k_2) \cosh(\theta_p \Delta t) + (q_1 k_2 + q_2 k_1) \sinh(\theta_p \Delta t)$$

Score completo = soma sobre os pares, com o envelope de recência $e^{-|\Delta t|\theta'}$ e o fator usual $1/\sqrt{d}$ ($\Delta t = t_i - t_j$):

$$s_{ij} = \frac{1}{\sqrt{d}} \sum_{p=1}^{d/2} e^{-|\Delta t|\theta'} s^{(p)}.$$

Passo 1. Substituímos cada um pela sua definição (cosh é par, então $\theta_p \Delta t \rightarrow \theta_p |\Delta t|$):

$$\cosh(\theta_p \Delta t) = \frac{1}{2} (e^{\theta_p |\Delta t|} + e^{-\theta_p |\Delta t|}), \quad \sinh(\theta_p \Delta t) = \frac{\text{sign}(\Delta t)}{2} (e^{\theta_p |\Delta t|} - e^{-\theta_p |\Delta t|}).$$

Passo 2. Multiplicamos o termo do cosh pelo envelope e *juntamos os expoentes* (esse é todo o truque):

$$\begin{aligned} e^{-|\Delta t| \theta'} \cosh(\theta_p \Delta t) &= \frac{1}{2} \left(e^{-|\Delta t| \theta'} e^{+\theta_p |\Delta t|} + e^{-|\Delta t| \theta'} e^{-\theta_p |\Delta t|} \right) \\ &= \frac{1}{2} \left(e^{-|\Delta t|(\theta' - \theta_p)} + e^{-|\Delta t|(\theta' + \theta_p)} \right). \end{aligned}$$

Passo 3. Como $\theta' > \max_p \theta_p$ por construção, tanto $\theta' - \theta_p > 0$ quanto $\theta' + \theta_p > 0$: **todo expoente é negativo**, então cada termo fica em $(0, 1]$, sem overflow, sem NaN. (O termo do sinh se junta da mesma forma, resultando em $\frac{\text{sign}(\Delta t)}{2} (e^{-|\Delta t|(\theta' - \theta_p)} - e^{-|\Delta t|(\theta' + \theta_p)}).$)

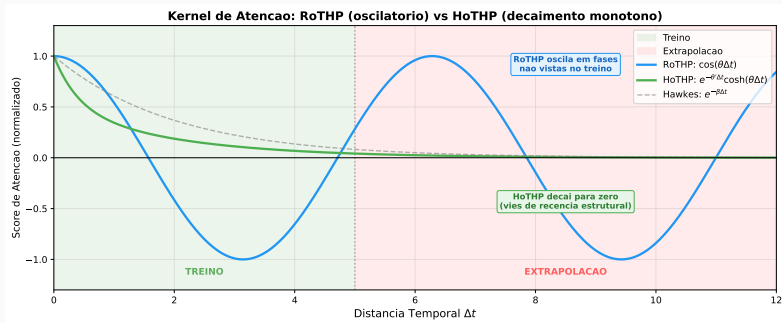
O HoTHP é um encoder Transformer com atenção multi-cabeça hiperbólica. **Nenhuma** informação de tempo é somada ao embedding; o tempo entra apenas pelo kernel. O viés atua em **duas etapas distintas**:

1. **Atenção hiperbólica** $\rightarrow \mathbf{h}(t_j)$: o decaimento decide *quais eventos passados* moldam o estado oculto. Eventos recentes recebem mais peso.
2. **Intensidade** $\lambda_k(t) = \text{softplus}(\alpha_k(t - t_j) + \mathbf{w}_k^\top \mathbf{h}(t_j) + b_k)$: linear no tempo *decorrido* $t - t_j$, então deslocar todos os instantes não muda nada; α_k é livre (pode subir ou descer após um evento).

Consequência

O viés de recência **não** exige que o processo seja auto-excitante. Exige apenas que eventos recentes sejam mais informativos. Ele ajuda em processos auto-excitantes e auto-inibitórios.

Kernel RoTHP vs. HoTHP: zonas de treino e extrapolação



Na **zona de treino**, os dois modelos ajustam os dados. Na **zona de extrapolação**, o kernel trigonométrico do RoTHP continua oscilando em fases não vistas, enquanto o kernel hiperbólico do HoTHP decai exponencialmente até zero, sem oscilação, como o kernel de Hawkes clássico (tracejado).

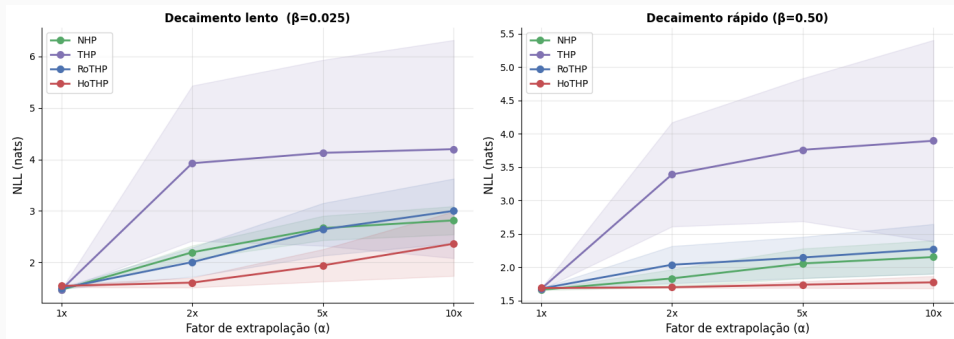
Experimentos e resultados

- **Dados sintéticos:** Hawkes multivariado com kernel exponencial, dois regimes: *decaimento lento* (β pequeno) e *decaimento rápido* (β grande). Conhecemos o processo gerador.
- **Dados reais (EasyTPP [9]):** Taxi, StackOverflow, Amazon, Retweet.
- **Protocolo:** treinar curto, testar longo, fatores $\alpha \in \{1, 2, 5, 10\}$ (sintético) e $\{1, 2, 5\}$ (real).
- **Modelos:** NHP, THP, RoTHP, HoTHP. THP/RoTHP/HoTHP compartilham a mesma arquitetura e intensidade, diferindo **apenas em como o tempo entra na atenção** (uma comparação controlada).
- **Métricas:** NLL (principal), RMSE, acurácia de tipo. Sintético: 10 sementes + teste de Wilcoxon (postos sinalizados). Real: 3 sementes (sem Wilcoxon).

Cenário	Modelo	1×	2×	5×	10×
Decaim. lento	NHP	1.470	2.197	2.667	<u>2.817</u>
	THP	<u>1.475</u>	3.926	4.127	4.199
	RoTHP	1.499	<u>2.007</u>	<u>2.642</u>	3.001
	HoTHP	1.545	1.609	1.943	2.364
Decaim. rápido	NHP	1.663	<u>1.833</u>	<u>2.057</u>	<u>2.153</u>
	THP	<u>1.677</u>	3.391	3.761	3.897
	RoTHP	1.681	2.039	2.146	2.275
	HoTHP	1.689	1.701	1.740	1.774

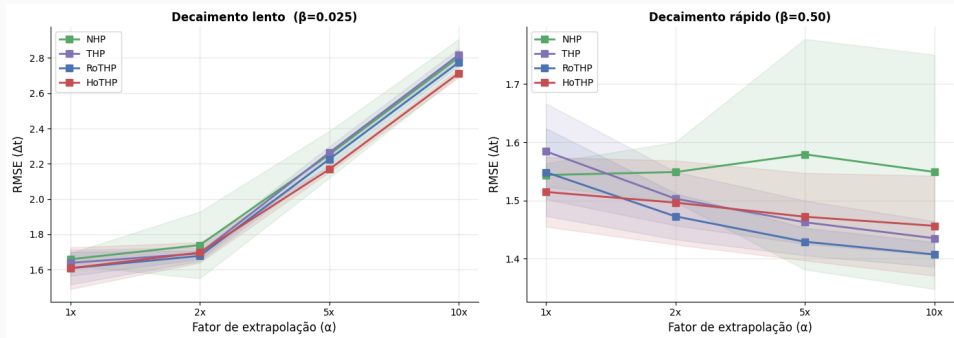
NLL (média, 10 sementes), menor é melhor. **O HoTHP vence em todos os fatores de extrapolação** (2×, 5×, 10×) nos dois cenários; o pequeno custo em 1× se reverte logo em seguida. Vantagem significativa (Wilcoxon, $p < 0.05$) em todos os casos.

Resultados sintéticos: NLL vs. fator de extrapolação



No decaimento rápido, a NLL do HoTHP cresce apenas 0.085 de $1\times$ a $10\times$ (quase plana), enquanto o THP diverge e RoTHP/NHP degradam mais. O viés de recência está alinhado com o kernel exponencial que gerou os dados.

Resultados sintéticos: RMSE



Diferente da NLL, o RMSE não separa os modelos: eles ficam próximos em todos os fatores. A vantagem do HoTHP aparece na **verossimilhança** (modelar a distribuição temporal completa), não na predição pontual do próximo intervalo.

Conjunto	Tipos	L_{train}
Amazon	16	18
StackOverflow	22	20
Taxi	10	7
Retweet	3	50

Instantes crus (apenas deslocados para zero), 3

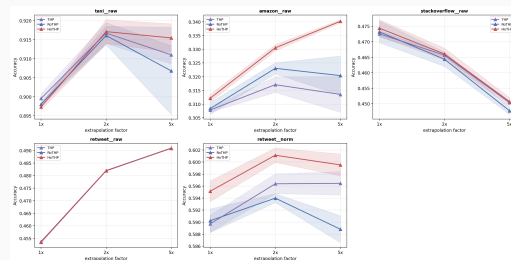
sementes. O Retweet é estudado à parte (ablação).

Conjunto	Modelo	1×	2×	5×
Taxi	THP	-0.273	-0.314	-0.243
	RoTHP	-0.271	-0.317	-0.137
	HoTHP	<u>-0.278</u>	<u>-0.328</u>	<u>-0.266</u>
	NHP	-0.449	-0.506	-0.460
Amazon	THP	2.264	2.237	2.254
	RoTHP	2.268	2.230	2.247
	HoTHP	<u>2.263</u>	<u>2.207</u>	<u>2.187</u>
	NHP	0.658	0.630	0.638
StackOv.	THP	<u>2.744</u>	<u>2.642</u>	2.519
	RoTHP	2.766	2.655	2.545
	HoTHP	2.758	2.649	2.536
	NHP	2.742	2.633	<u>2.524</u>

Dentro da família Transformer, o HoTHP tem a melhor NLL na Amazon e no Taxi em todos os fatores, e é o único que continua *melhorando* na Amazon conforme a sequência cresce.

Dados reais: acurácia de predição de tipo

Conjunto	Modelo	1×	2×	5×
Amazon	NHP	.293	.306	.315
	THP	.308	.317	.314
	RoTHP	.308	.323	.320
	HoTHP	.312	.331	.340
StackOv.	NHP	.469	.462	.445
	THP	.473	.466	.450
	RoTHP	.473	.464	.448
	HoTHP	.474	.466	.451
Retweet [†]	NHP	—	—	—
	THP	.590	.596	.597
	RoTHP	.590	.594	.589
	HoTHP	.595	.601	.600

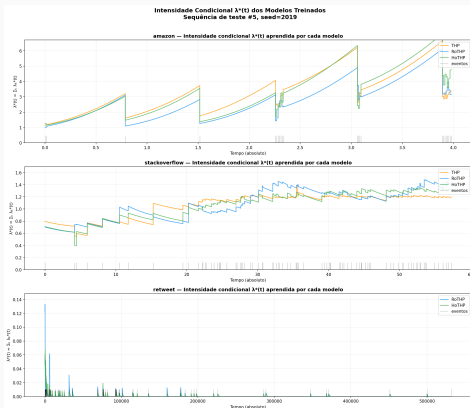


† Retweet normalizado (instantes crus estouram todos os

Transformers); NHP não foi rodado nele.

O HoTHP tem a **melhor acurácia em todos os fatores, em todo conjunto onde a recência ajuda**. Ele supera a família Transformer e também o NHP, cuja vantagem se restringia à NLL. Na Amazon a margem *cresce* com o comprimento da sequência (+0.020 sobre a melhor baseline em 5×).

O que os modelos aprendem: intensidades condicionais



$\lambda^*(t)$ aprendido em sequências de teste:

- **Amazon:** intensidade cresce entre eventos e cai a cada um (*auto-inibitório*). Ainda assim o HoTSP vence.
- **StackOverflow:** intensidade quase plana (pouca memória); os modelos empatam.
- **Retweet:** reagem só à rajada inicial e colapsam para $\lambda^* \approx 0$ (escala extrema).

Confirma a visão em duas etapas

O viés atua na **atenção** (quais eventos importam), não no sinal da intensidade. Por isso ajuda até em processos auto-inibitórios.

Retweet: escala ou viés?

Versão	Modelo	1×	5×
Cru	THP	15.9	2.3×10^4
	RoTHP	15.9	7.0×10^5
	HoTHP	15.9	1.4×10^5
Norm.	THP	-2.180	<u>0.039</u>
	RoTHP	-2.133	47.6
	HoTHP	<u>-2.139</u>	-0.651

O problema era a **escala**. Após a normalização, o HoTHP é o único modelo com NLL negativa (estável) em 5×.

Custo vs. benefício

- O HoTHP calcula o kernel por *par* de eventos: $O(L^2)$.
- Nos dados reais ele é de $1.07\times$ (Taxi) a $2.44\times$ (Retweet) mais lento que o RoTHP, mas **mais barato que o NHP** (a baseline mais forte em NLL).
- O custo é pago uma vez, no treino. Na inferência, o que importa é um resultado *válido*: o RoTHP em 5× dá NLL 47.6, o HoTHP dá -0.651.

Conclusão

Seguindo o retorno da banca (qualificação, 10 de julho de 2026), a dissertação foi revisada em quatro frentes:

- **Notação e legibilidade:** siglas expandidas no resumo, lista de símbolos adicionada, notação unificada (γ_k para a inclinação da intensidade, ρ para o fator de extrapolação) e o módulo corrigido na equação do kernel ($e^{-|\Delta t|\theta'}$).
- **Fundamentação mais profunda:** uma taxonomia Poisson–Hawkes, a ligação explícita de verossimilhança Poisson $\rightarrow$ Hawkes e uma discussão sobre a limitação de período do RoPE.
- **Mais resultados:** RMSE agora reportado nos conjuntos reais, e fontes adicionadas a toda figura reaproveitada.
- **Escrita:** a conclusão e o capítulo de trabalhos futuros concluídos.

- **HoTHP**: um Transformer Hawkes Process com kernel rotativo **hiperbólico**, combinando invariância no tempo com um viés de recência exponencial estrutural em um único mecanismo de atenção.
- Um θ' aprendível, parametrizado de modo que $\theta' > \max_j \theta_j$ **por construção**, garantindo o decaimento exponencial do score e estabilidade numérica.
- Sob *treinar curto, testar longo*, o HoTHP mantém a NLL estável até $10\times$ nos dados sintéticos (significativo por Wilcoxon), e é o melhor da família Transformer na Amazon e no Taxi.
- Uma análise separando *onde* o viés atua (atenção vs. intensidade), explicando por que ele ajuda mesmo em processos auto-inibitórios.
- Implementação e experimentos no framework EasyTPP.

Limitações

- O viés exponencial atrapalha quando eventos distantes são tão relevantes quanto os recentes (memória de longo alcance).
- Escalas temporais extremas precisam de normalização para evitar saturação numérica (como no Retweet cru).
- θ' é uma taxa de decaimento **global única**; processos com múltiplas escalas temporais podem exigir múltiplas taxas.

Trabalhos futuros

- **Múltiplas escalas de decaimento:** um θ' aprendível por cabeça (ao estilo do ALiBi), cada uma se especializando em uma escala temporal.
- **Formas alternativas de decaimento:** lei de potência em vez de exponencial, para memória de longo alcance (réplicas sísmicas, cascatas).
- **Avaliação mais ampla:** benchmarks de horizonte longo (HoTPP), novos domínios.
- **Além da atenção:** o viés de recência se transfere para modelos de espaço de estados (Mamba) e TPPs baseados em LLM?

Hugo Ramos Soares

Universidade Federal do Ceará

Orientador: César Lincoln C. Mattos

Coorientador: João Paulo Pordeus Gomes

- [1] Chang Dai, Hongyu Shan, Mingyang Song, and Di Liang.
Hope: Hyperbolic rotary positional encoding for stable long-range dependency modeling in large language models.
2025.
- [2] Shan Dai, Anningzhe Gao, Zhuo Li, and Yuhao Du.
Rotary position embedding-based transformer hawkes process for event-type big data.
Big Data Mining and Analytics, 9(1):23–38, 2026.
- [3] Alan G. Hawkes.
Spectra of some self-exciting and mutually exciting point processes.
Biometrika, 58(1):83–90, 1971.
- [4] Hongyuan Mei and Jason Eisner.
The neural Hawkes process: A neurally self-modulating multivariate point process.
In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [5] Ofir Press, Noah A. Smith, and Mike Lewis.
Train short, test long: Attention with linear biases enables input length extrapolation.
In *International Conference on Learning Representations (ICLR)*, 2022.
- [6] Sheldon M. Ross.
A First Course in Probability.
Pearson, Boston, MA, 10th edition, 2018.
- [7] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu.
Roformer: Enhanced transformer with rotary position embedding.
Neurocomputing, 568, 2024.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [9] Siqiao Xue, Xiaoming Shi, Zhixuan Chu, Yan Wang, Hongyan Hao, Fan Zhou, Caigao Jiang, Chen Pan, James Y. Zhang, Qingsong Wen, Jun Zhou, and Hongyuan Mei.
EasyTPP: Towards open benchmarking temporal point processes.
In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Simiao Zuo, Haoming Jiang, Zichong Li, Tuo Zhao, and Hongyuan Zha.
Transformer hawkes process.
In *International Conference on Machine Learning (ICML)*, 2020.

Backup: por que hiperbólico, e não um único $e^{-\beta|\Delta t|}$ ou ALiBi?

Expandindo o score, obtemos uma **mistura de exponenciais ponderada pelo conteúdo**:

$$s_{ij} = \frac{1}{\sqrt{d}} \sum_{p=1}^{d/2} \left[c_p(\mathbf{q}, \mathbf{k}) e^{-(\theta' - \theta_p)|\Delta t|} + d_p(\mathbf{q}, \mathbf{k}) e^{-(\theta' + \theta_p)|\Delta t|} \right]$$

Três coisas que um único $e^{-\beta|\Delta t|}$ ou um viés tipo ALiBi **não** dão:

1. **Coeficientes dependentes do conteúdo** c_p, d_p (os $\mathbf{q} \cdot \mathbf{k}$ produtos por par de dimensões), que podem ser positivos ou negativos; a penalidade do ALiBi é fixa e independente do conteúdo.
2. **Um espectro de taxas de decaimento** $\{\theta' - \theta_p, \theta' + \theta_p\}$: como θ_p varia de 1 até $\sim 10^{-4}$, componentes lentas ($\theta' - \theta_1$, perto de zero) retêm memória mais longa, enquanto componentes rápidas ($\theta' + \theta_p$) esquecem depressa. Um único β tem uma só escala.
3. **Invariância no tempo** (o score depende apenas de Δt), que o ALiBi discreto não oferece em tempo contínuo.

Backup: a família Transformer é competitiva? (NHP vs THP)

O NHP vence em NLL na Amazon/Taxi: é esperado, não um bug. Isso condiz com o benchmark do EasyTPP: modelos recorrentes em tempo contínuo modelam a densidade entre eventos de forma nativa. Nossa pergunta é *dentro da família* (arquitetura fixa, só o kernel muda): o HoTHP é o melhor dos três, supera o NHP em **acurácia** (Amazon, SO) e supera *todas* as baselines na NLL sintética (10 sementes, Wilcoxon).

Também investigamos a inversão $NHP > THP$ diretamente

Reproduzindo o THP **original** nos próprios dados MIMIC-II dele:

- Sob uma rotina de integração consistente, $NHP \geq THP$ mesmo ali.
- A LL alta do artigo do THP vem da variante de solver Monte Carlo: o efeito de **implementação** na LL (≈ 2.76 nats) é $\sim 10\times$ o de **modelo** (≈ 0.28 nats).

Ou seja, LLs de implementações diferentes não são comparáveis; a inversão não é artefato da nossa reimplementação.

(Investigação complementar, em uma base de código separada; não é um capítulo da dissertação.)

Backup: para quanto o θ' convergiu?

Parametrizado de modo que $\theta' = \text{softplus}(\text{raw}) + 1 + 10^{-4} > \max_p \theta_p = 1$ **por construção**.

Cenário / conjunto	θ' aprendido
Sintético, decaim. rápido	[preencher]
Sintético, decaim. lento	[preencher]
Amazon / Taxi / StackOverflow	[preencher]

Expectativa: no *decaimento rápido*, θ' maior (memória mais curta); no *decaimento lento*, θ' menor.

TODO antes da defesa: extrair $\theta' = \text{softplus}(\text{raw}) + 1 + 10^{-4}$ dos checkpoints e preencher a tabela.

Backup: como a integral do compensador é calculada?

- A **mesma rotina** para os quatro modelos, definida uma única vez na classe base do EasyTPP.
- Uma **grade determinística** de 20 nós por intervalo entre eventos (uma quadratura `linspace` fixa; o texto a chama de Monte Carlo).
- **Ponto central:** qualquer viés do estimador é *idêntico* entre os modelos, então o ranking de NLL não depende da aproximação da integral.